Data Collections part 2: More on Dictionaries

CMSC 104 Section 2
November 2, 2026

Administrative Notes

Classwork 7 is due Wednesday, November 4

Project 3 will be delayed by one week. It was originally going to be assigned today, and due on November 16. Now it will be assigned next Monday, November 9 and due on Monday, November 23.

- The reason for the delay is that it requires reading in files, which we will not cover until Wednesday and next Monday

Quiz 3 is scheduled for next Wednesday, November 11

More on dictionaries:
stuff we didn’t get to on Monday

When would you use a dictionary?

Use a list if you have an ordered collection of items - a list of student IDs; a list of grades; etc.

Use a dictionary if you have a set of unique keys that each map to a value.

The dog example - we have a set of dog names (‘keys’) that each map to one or more breeds of dogs (‘values’). It would be awkward to use lists to store dog names and breeds and maintain the relationship.

- You could do it, but it would be awkward

So how do you access elements of a dictionary?

By key value only. Put the key in square brackets:

dogs[‘doug’] gets you the list [‘beagle’,’bulldog’]

dogs[‘remy’] gets the string ‘shepherd’

What if that key’s not present?

dogs[‘pepper’]

Operations on dictionaries are easy ways to create bugs that crash your program. “Defensive programming” is in order - use methods that are resilient to bugs.

The ***get*** method

Another way to access a dictionary element that's more resilient to errors

dogs.get('pepper')

If that key doesn't exist, the method will return None (that special value we talked about) instead of erroring out. So your program can continue rather than crashing.

You can make it return something other than None, if you want

dogs.get('pepper','Key not found')

dogs.get('pepper', -1)

Removing an element from a dictionary

```
del dict[key]
```

In the dogs example:

```
  del dogs['cinder']
```

Removes the entire key/value pair associated with ‘cinder’

But again, if that key doesn’t exist in the dictionary, you’ll error out. Your program may crash.

```
   del dogs['lady']


if 'lady' in dogs.keys():
    del dogs['lady']  # gets rid of key and value pair
```

A more resilient way to delete - the pop method

```
dogs.pop('lady')
```

Still causes an error if ‘lady’ is not a key in the dictionary

But you can specify a default value to return if the key isn’t there

```
dogs.pop('lady', None)
```

OR

```
dogs.pop('lady', -1)
```

etc.

Creating a dictionary from two lists

Python “zip” function and “dict” constructor method

tl;dr:

```
 a = [1,2,3]
 b = [4,5,6]
 d = dict(zip(a,b))
 print(d)
[1:4, 2:5, 3:6]
```

The longer story: “zip” is a function that takes multiple iterable values (lists) and interweaves them, one element at a time. One element from list a, then one from list b, then the next one from a, then from b… until the shortest list is done. The return value from zip() is an iterator that can be used to build other structures. The return value from zip() is not meaningful to the average human.

“dict” is a constructor method that operates on an iterator and returns a dictionary. It’s analogous to the “list” method that creates lists.

Here, “zip(a,b)” weaves together the values 1,4,2,5,3,6 and returns an iterator built from them. Then “dict(zip(a,b))” takes that iterator and turns it into the dictionary you see. The first item becomes a key; the second its associated value. The third item becomes the second key; the fourth is its associated value. And so on.

Searching for a value …

We can use .values() and in to see if a particular value is in the dictionary

…and then use a brute-force search to identify what key is mapped to that value

But that doesn’t work if your values include lists and/or dictionaries!

Really: when would you use a dictionary?

When you want to keep all information about an object together, and there are different types of information

An example:

Suppose you want to collect information about this year’s Nobel Prize winners?

A first skeleton of our dictionary

We know the categories, but don’t know the winners:

```
# this file creates a dictionary of the 2022 Nobel Prize winners

# keys will be the categories;
# in the initial version values are the names of the winners

# later we'll produce a more complicated version

nobels = {
    "medicine": None,
    "chemistry": None,
    "physics": None,
    "literature": None,
    "peace": None,
    "economics": None

}
```

The 2026 winners:

physiology or medicine:

Physics:

Chemistry:

Literature:

Peace:

economic science:

Nobels[“medicine”] =
Nobels[“physics’} =

Lists of dictionaries

Note: this is relevant to project 3

Suppose you created a dictionary of Nobel winners for this year

- And you also want dictionaries for the 2025 winners; 2024 winners; and so on.
- You can make a list of these dictionaries
- You could also make a dictionary of these dictionaries - why would a list be better?

JSON

Java Script Object Notation - a common file format for exchanging information on the Internet

- Here’s a hint: it’s essentially a list of dictionaries